

KOTLIN

COMPLETE NOTES

Easy Language • Simple Examples • Important Topics Highlighted

★ Red star = Important Topic (read carefully, exam / interview favorite)

MMOC TEACH
MADE BY PRAVENDRA

INDEX

★ 1. Introduction to Kotlin

★ 2. Variables - val vs var

- 3. Data Types
- 4. Conditional Statements (if as an Expression)
- 5. Loops - for, while

★ 6. Functions

★ 7. Null Safety

★ 8. Classes and Objects

★ 9. Data Classes

- 10. Inheritance
- 11. Collections - List and Map
- 12. Lambda Expressions
- 13. Extension Functions

★ 14. Coroutines (Basics)

★ 1. Introduction to Kotlin *[IMPORTANT TOPIC]*

Explanation:

Kotlin is a modern programming language, officially recommended by Google for building Android apps. It is designed to be shorter, safer, and easier to read than Java, while still working perfectly with existing Java code.

Key Points:

- Official language for Android app development.
- Shorter and cleaner code compared to Java.
- Has built-in Null Safety, which helps avoid a very common type of app crash.

Example Code:

```
fun main() {  
    println("Hello, World!")  
}
```

Output:

Hello, World!

 *Note: fun is the keyword used to define a function in Kotlin.*

★ 2. Variables - val vs var [IMPORTANT TOPIC]

Explanation:

Kotlin has two ways to create a variable. 'val' creates a value that CANNOT be changed after being set (like a constant). 'var' creates a variable whose value CAN be changed later.

Example Code:

```
val name = "Pravendra" // cannot be changed
var age = 21           // can be changed

age = 22
println("$name is $age years old")
```

Output:

```
Pravendra is 22 years old
```

💡 *Note: Always prefer 'val' unless you are sure the value needs to change — it makes code safer.*

3. Data Types

Explanation:

Kotlin has clear, simple data types, and it can often automatically detect a variable's type without us writing it directly.

Key Points:

- Int -> whole numbers
- Double -> decimal numbers
- String -> text
- Boolean -> true or false

Example Code:

```
val age: Int = 21
val height: Double = 5.9
val isStudent: Boolean = true
println(age)
```

Output:

```
21
```

4. Conditional Statements (if as an Expression)

Explanation:

Kotlin's 'if' can be used as a normal conditional statement, but interestingly it can also directly RETURN a value, unlike many other languages.

Example Code:

```
val marks = 75

val grade = if (marks >= 90) "A" else if (marks >= 60) "B" else "C"
println("Grade: $grade")
```

Output:

Grade: B

5. Loops - for, while

Explanation:

Loops repeat a block of code multiple times. Kotlin's 'for' loop commonly uses ranges, like 1..5, to repeat a fixed number of times.

Example Code:

```
for (i in 1..3) {  
    println("Count: $i")  
}
```

Output:

```
Count: 1  
Count: 2  
Count: 3
```

★ 6. Functions *[IMPORTANT TOPIC]*

Explanation:

A function is a reusable block of code, created using the 'fun' keyword. Kotlin functions can also have default parameter values.

Example Code:

```
fun addNumbers(a: Int, b: Int): Int {  
    return a + b  
}  
  
println(addNumbers(5, 7))
```

Output:

12

★ 7. Null Safety *[IMPORTANT TOPIC]*

Explanation:

Null Safety is one of Kotlin's most important features. By default, a normal variable CANNOT be null (empty). To allow a variable to possibly be null, we must mark its type with a ? symbol. This prevents a very common app-crashing error called a 'NullPointerException'.

Example Code:

```
var name: String = "Pravendra"  
// name = null    // ERROR - not allowed, since name cannot be null  
  
var nickname: String? = null    // allowed, marked with ?  
println(nickname)
```

Output:
null

💡 *Note: The ? symbol after a type is Kotlin's way of saying 'this value is allowed to be null'.*

★ 8. Classes and Objects *[IMPORTANT TOPIC]*

Explanation:

A class is a blueprint for creating objects, which group data and behavior together. Kotlin classes are shorter to write than in many other languages.

Example Code:

```
class Car(val brand: String) {  
    fun showBrand() {  
        println("Brand: $brand")  
    }  
}  
  
val c1 = Car("Toyota")  
c1.showBrand()
```

Output:

Brand: Toyota

★ 9. Data Classes *[IMPORTANT TOPIC]*

Explanation:

A data class is a special kind of class in Kotlin, made specifically to hold data. Kotlin automatically generates useful code for it, like printing and comparing, saving us from writing repetitive code.

Example Code:

```
data class Student(val name: String, val age: Int)

val s1 = Student("Pravendra", 21)
println(s1)
```

Output:

```
Student(name=Pravendra, age=21)
```

 *Note: Without 'data class', printing an object like this would show a confusing memory address instead of readable values.*

10. Inheritance

Explanation:

Inheritance lets one class reuse the properties and functions of another class. In Kotlin, a class must be marked 'open' before it can be inherited from — this is different from many other languages.

Example Code:

```
open class Animal {  
    fun eat() { println("This animal eats food") }  
}  
  
class Dog : Animal() {  
    fun bark() { println("The dog barks") }  
}  
  
val d = Dog()  
d.eat()  
d.bark()
```

Output:

```
This animal eats food  
The dog barks
```

11. Collections - List and Map

Explanation:

A List stores an ordered group of items. A Map stores data as key-value pairs, similar to a dictionary. Kotlin collections can be 'immutable' (fixed, cannot change) or 'mutable' (can be changed).

Example Code:

```
val fruits = listOf("apple", "banana", "mango")
println(fruits[0])

val marks = mapOf("Pravendra" to 90, "Aman" to 85)
println(marks["Pravendra"])
```

Output:

```
apple
90
```

12. Lambda Expressions

Explanation:

A lambda is a short, unnamed function that can be written in a single line and passed around like a value. They are commonly used with collection functions.

Example Code:

```
val numbers = listOf(1, 2, 3, 4)
val doubled = numbers.map { n -> n * 2 }
println(doubled)
```

Output:

```
[2, 4, 6, 8]
```

13. Extension Functions

Explanation:

An extension function lets us ADD a new function to an existing class (even one we did not create), without changing its original code. This is a unique and powerful Kotlin feature.

Example Code:

```
fun String.addExclamation(): String {  
    return this + "!"  
}  
  
val greeting = "Hello".addExclamation()  
println(greeting)
```

Output:
Hello!

★ 14. Coroutines (Basics) *[IMPORTANT TOPIC]*

Explanation:

Coroutines are Kotlin's way of running tasks (like fetching data from the internet) in the background, WITHOUT freezing the app, and without the complexity of traditional multi-threading.

Example Code:

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        delay(1000L)    // waits 1 second, without freezing the app
        println("Data loaded!")
    }
    println("Loading...")
}
```

Output:

```
Loading...
Data loaded! (shown after 1 second)
```

 *Note: Coroutines are heavily used in real Android apps for tasks like network calls and database access.*